Uncle Bernie's
Apple - 1 Color Graphics Card
2nd Generation Version
Documentation for Arnaud

Note to Arnaud: this is a document I made specifically for you for upgrading your POM-1. Seems you succeeded to nudge me into writing it up … it was a lot of work, but I will turn it into the "User Manual" for the graphics card later. I am very excited that you volunteered to fully implement my graphics card in your POM-1, as this will be instrumental to make our work available (and attractive) to a much, much larger user community. We should work together to make POM-1 the first and foremost Apple-1 emulator there is ! Once it has the graphics card emulation, I will use it for development of the software library which is needed for the card. But I need a LINUX version of POM-1 which runs on Linux Mint 19 or 22.

TECHNICAL SPECIFICATIONS (of the graphics card)

- RAM expansion to 54 kBytes: $0000-$BFFF plus the $E000-$EFFF on motherboard
- TEXT mode, 40 x 25 characters, black-and-white, lowercase characters available
- LORES graphics mode, 40 x 50 pixels of 16 colors
- HIRES graphics mode, 280 x 192 pixels of 6 colors
- MIXED graphics / text mode
- CPU can read horizontal and vertical blank flag
- NTSC conforming color signal (PAL not supported)
- selectable vertical frequency 50 Hz or 60 Hz
- power consumption: 2.3W (5V / 0.46A) with GALs, 3.7W (5V / 0.74A) with MMI PALs
- physical size: XXmm x XXmm (short version), XXmm x XXmm (long version with fan)
- optional fan to cool the LM323K regulator on the motherboard (on long PCB version only)

The screen buffer memory pages and the soft switches for selection of the graphics mode work much like in the Apple II, with the exception that the addresses of the soft switches have been moved from $C050-$C057 to $C250-$C257 to avoid address conflicts with the "Apple Cassette Interface" (ACI).

The "vaporlock" programming technique known from the Apple II is not supported, due to the ACI occupying all the memory locations where "vaporlock" would read the "bit vapors". As a replacement, the horizontal and vertical blank flag HST0 was provided which can be read by the software.

PROGRAMMING TIPS & TRICKS

The graphics card soft switches had to be moved to a different address page and they now are "read only", which means they don't react when being written to. Most Apple II games "flick" these soft switches by reading them, so there should be no issue with that change whose purpose is to avoid data bus clashes with the HST0 flag readout and still allow it to be read on every soft switch location.

All graphics card soft switches have mirror addresses due to limited number of address lines being decoded. They repeat every 8 locations throughout the $C2xx, $C3xx, $C6xx and $C7xx memory

pages with the additional rule that A4 must be 1. This was done to reserve lots of soft switch addresses for future Apple-1 expansion cards. The soft switch addresses for the graphics card must have A4 set, so the third nibble of their hex address must be one of the following: 1, 3, 5, 7, 9, B, D, F.

For Arnaud specifically: the address decoder equation is:

SEL = $Cxxx & !A11 & A9 & A4;

The following are the recommended addresses to be used in software. But the POM-1 must decode the addresses using the above equation and only react in read cycles to change the soft switches. And put the HST0 flag into D7 (MSB). I would recommend to randomize the other bits so as to thwart stupid attempts by rookie programmers. Sounds unfair, but will help to get them on the right track ;-)

Soft Switch	Address	Symbol	Action
SS_TEXT	$C250	TEXT_OFF	Turns TEXT mode off, enables all other modes
	$C251	TEXT_ON	Turns TEXT mode on, disables all other modes
SS_MIX	$C252	MIX_OFF	Full graphics screen
	$C253	MIX_ON	Split screen, last four lines are TEXT mode
SS_PAGE	$C254	PAGE_ONE	Select primary page ($400 - $7FF, HIRES: $2000 - $3FFF)
	$C255	PAGE_TWO	Select secondary page ($800 - $FFF, HIRES: $4000 - $5FFF)
SS_HIRES	$C256	LORES_ON	LORES graphics mode (only if SS_TEXT is turned off)
	$C257	HIRES_ON	HIRES graphics mode (only if SS_TEXT is turned off)

Table 1: Screen mode soft switches and their actions

When porting Apple II games, just identify all the soft switch and port accesses in the code, and change the H-Byte of all screen mode soft switches from $C0 to $C2. Neutralize (replace) all other soft switch accesses except for $C030-$C03F, which in the Apple II toggles the SPEAKER output. Leave these intact because then the ACI will respond and produce the same sound effects as the Apple II would, when an amplifier and loudspeaker is plugged into its TAPE OUT jack.

Note to Arnaud: this means: in POM-1, you do not need to change any code for the ACI but you could add Apple II style sound to it, which would use the TAPE OUT state bit in lieu of the SPEAKER bit. It's more tricky to get the sound sounding right, but you can adopt the finely crafted code from various good Apple II emulators. What I'd like is to port the "song" section from Apple II GHOSTBUSTERS to the Apple-1. My upcoming "Uncle Bernie's Apple-1 game system" will plug into TAPE OUT and do exactly the same thing. I also have some speech synthesis based on Forrest Mozer's patents as I was somewhat involved in reverse engineering that TSI chip. I would love to port that from "C" to the Apple-1. Just for the chess playing programs I intend to adopt for it … don't tell anyone.

At the time of this writing, no joystick inputs are available for the Apple-1. No big issue as many Apple II games support keyboard inputs to games. Early Apple II games had no joystick support anyways, and could be played just fine with only the keyboard, similar to playing "DOOM" on the PC. The best way to deal with this limitiation right now is to route all "joystick" code through a subroutine which later can be upgraded to joystick support, once the hardware to interface joysticks is available.

Note to Arnaud: this is a future project. I intend to use the TAPE IN of the ACI to interface the joysticks. But other than some brainstorm sketches on paper, there is nothing designed and built yet. But be prepared to have USB joystick support in POM-1. The code can be taken from Linapple.

Vaporlock

"Vaporlock" code of Apple II games must be identified and eliminated. While it is possible to leave all the writes to the "screen holes" intact, the interrogation loop which typically precedes a screen page flip using the SS_PAGE soft switch must be replaced with a subroutine call that interrogates the horizontal and vertical blank flag HST0 and identifies the vertical blank period.

VBLANK detection using the HST0 flag

Here is an example how to code the VBLANK detection:

```
SS_PAGE  EQU $C254

;
; ------ VBLANK period detection subroutine with page flip ------
;
; On entry: Y set to active screen page (primary:0 secondary:1)
;
; On exit:  if CY = 1, screen page was flipped, new page in Y.
;           if CY = 0, not yet in VBLANK. Y was not changed.
;           In both cases, A and X were changed.

VBLDET   LDX #2        ; three trips through the loop needed
VBLOOP   LDA SS_PAGE,Y ; first  sample
         ORA SS_PAGE,Y ; second sample, 4 cycles later, mask burst
         BPL NOTVBL    ; taken if electron beam not in vblank
         DEX           ; decrement loop counter
         BPL VBLOOP    ; continue loop
      ;
      ; beam is in VBLANK
      ;
         INY           ; change Y to the other page
         TYA
         AND #1
         TAY
         LDA SS_PAGE,Y ; flip the page
         SEC
         RTS
      ;
      ; not in VBLANK
      ;

NOTVLB   CLC
         RTS
```

Listing 1: how to detect VBLANK

Note that the HST0 flag is not the same as the VBI flag of the Apple IIc and Apple IIe, as HST0 also contains some other timing information about the horizontal blanking period, such as where the color

burst signal is. This makes interpretation of the MSB more difficult than on the Apple IIc or IIe, but it also gives use the opportunity to actually count on which scan line the electron beam is and then split the screen / change the graphics mode on-the-fly, anywhere we want to do that. More about the HST0 flag in APPENDIX 1.

Be aware that the detection method shown in the example code above is not 100% perfect in all use cases because there is a weakness when VBLDET is called just at a point in time near the end of the VBLANK period, where the VBLOOP takes its last sample of HST0 at the very end of the VBLANK. The page flip will occur about 22 CPU cycles after the last sample. Which may be in the mid of the very first scan line of the active picture. If any moving objects in that half of the very first scan line have been changed from the previous page to the new page, there may be a brief disturbance, almost imperceptible to the human eye. So, if you choose to have moving objects crossing the upper screen boundary, either blank them out in the first screen line (easy, in your sprite rendering code just remove the STA command for the first scan line which would put any sprite / moving object there) or design your main game loop such that it will have completed drawing all the moving objects before the VBLANK period begins. Then you can check for VBLANK being off and after that wait for the begin of VBLANK before doing the page flip. After that flip at the beginning of VBLANK has been done, it is known that ~4200 CPU cycles remain to draw further objects in the newly activated screen page before it gets “live” and the electron beam starts to write it on the screen.

Character sets

Uncle Bernie’s Apple - 1 Color Graphics Card uses a type 2716 EPROM which unlike the early Apple II and II+ contains the full ASCII character set in the same way the later Apple IIe does. When the type 2716 EPROM is replaced with a type 2732 EPROM, a manual switch could be added to choose between two character sets, which would allow to introduce country specific characters (such as German “Umlauts” and French “accents”) much in the same way as has been done back in the day.

Like in the Apple II, some character subsets can be displayed inverted or flashing, which are attributes controlled bt the upper two bits of the character code. This has been the same since the first Apple II that appeared in Y1977 and it only leaves six bits to choose one of 64 characters supported by the Signetics 2513 character generator which was used in both the Apple-1 and the early Apple II. The inevitable result is a peculiar way how the characters in the TEXT screen buffers must be encoded – it is not ASCII but the mapping is trivial:

Character code in screen buffer	Bit 7	Bit 6	Bit 5	Display Mode	Character Subset
$00 – $1F	0	0	0	inverted	@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_
$20 – $3F	0	0	1	inverted	!”#$%&’()*+,-./0123456789:;<=>?
$40 – $5F	0	1	0	flashing	@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_
$60 – $7F	0	1	1	flashing	!”#$%&’()*+,-./0123456789:;<=>?
$80 – $9F	1	0	0	normal	@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_
$A0 – $BF	1	0	1	normal	!”#$%&’()*+,-./0123456789:;<=>?
$C0 – $DF	1	1	0	normal	@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_

$E0 – $FF	1	1	1	normal	`abcdefghijklmnopqrstuvwxyz{|}~▒

Table 2: Character subsets and their display modes

Due to the origins of this encoding convention (the Signetics 2513 character generator), the encoding of the lower six bits of any ASCII character is the same for the Apple-1 and the Apple II, but the attribute bits and the lower case characters do not exist in the Apple-1, as its shift register based screen memory is only six bits wide. In other words, the Apple-1 ignores the two upper bits, while the Apple II uses them as attribute bits to control inversion and flashing.

Note to Arnaud: I can send you the character set template I use. This is a text graphical representation which can easily be parsed and turned into a binary.

APPENDIX 1: HOW THE HST0 FLAG WORKS

HST0 is one of the two state bits of the “horizontal timing state machine” within the graphics card. It is active one whenever the video signal is in either the horizontal blank or the vertical blank period, with one exception, which is the color burst period, during which HST0 is zero, despite this period belongs to the horizontal blank. The color burst period lasts for three CPU cycles, so a simple double sample technique which forms the logical OR of the two samples will eliminate this exception as far as the code is concerned.

Note that unlike the VBI flag of the Apple IIe and IIc which can be read at $C019 in these IOU custom chip based machines, the HST0 flag available when reading any soft switch address of this graphcis card conveys both horizontal blank and vertical blank information which makes it much more powerful than Apple’s VBI flag, but also more difficult to use. This is why a completely worked out solution for VBLANK detection has been given in Listing 1 above.

The CPU cycle timing for HST0 corresponds to the 65 CPU cycles per scan line and is the same for each of the 262 scan lines per field in the 60 Hz setting. The graphics card can be set to 50 Hz which may be helpful when using a low quality monitor in places with 50 Hz line frequency. In this case, there are 312 scan lines per field. But this setting of the graphics card is not recommended unless there are 10 Hz “hum bars” visible in the picture, or if the picture wobbles with 10 Hz, which should never happen with a fully functional, good quality TV or monitor. Software for the graphics card should be written such that it works both with the 50 Hz and 60 Hz settings. It is possible to count CPU cycles from VBLANK to VBLANK period to determine the setting of the graphics card. Hence, Apple-1 emulators should have the option to set the graphics card emulation to either case.

A behavioral model for HST0

Assuming that the emulated video scanner keeps track of the scan line number <line> and the horizontal counter <hcnt>, which for each scan line runs from [0:64] and then rolls over to 0, the following “C” function will return the state of the HST0 flag as it can be read by the 6502:

```
int hst0_state(int line, int hcnt)
{
 if((hcnt > 12) && (hcnt < 16)) return 0; // in the BURST period
 if(line > 191) return 1;                 // in VBLANK
 if(hcnt > 24) return 0;                  // in live scan
 return 1;                                // in HBLANK
}
```

Listing 2: behavioral model for HST0

This assumes that all lines from 0 to 191 are the visible “live scan” lines and that line 192 and above are the vertical blank period, which varies in length depending on whether it’s the 60Hz or 50Hz setting: for the former, the <line> variable would count up to and including 261 and then roll over to 0, and for the latter, it would count up to and including 311 and then roll over to 0.

For the horizontal counter, <hcnt> values from 25 to 64 correspond to the video byte positions 0 to 39 in the screen buffer.

Complications

The Apple-1 itself also has 65 CPU cycles per scan line. But four of these CPU cycles are “stolen” by the DRAM refresh. This means that any timing loop based on CPU cycle counting will miscount by the factor 61 / 65 = 0.938, meaning that when attempting to measure the vertical frequency, it will count less CPU cycles than expected from the ideal code. The other way around, if cycle counting is used to split the screen at other than the MIXED mode case supported by the hardware itself, then these refresh cycles will throw the timing off. This can be only partially compensated for by writing 6502 code which actually counts the HBLANK periods in the HST0 flag. For vertical screen splits, the cycle counting uncertainty must be further compensated by allowing enough “gap” vertical bands in which the screen split actually may happen by flicking a soft switch. For switches between HIRES and LORES graphics, this “gap” can be of any solid color, when appropriate bit patterns for both the HIRES case and the LORES case are positioned in the “gap”: LORES can produce the same colors as HIRES, although only at a reduced resolution, which can easily be matched in HIRES. So a uniformly colored “gap” of sufficient width is possible (what is “sufficient” is still t.b.d.). The same principle can be applied for splits between HIRES and TEXT modes, which involves rendering the same text in the HIRES screen buffer, for all those text character positions where the SS_TEXT soft switch may just turn on, under the cycle count uncertainty caused by the refresh cycles. Note that at the end of each such split screen scan line, the SS_TEXT soft switch must be turned off again, to enable the color burst signal. Otherwise, the next scan line might revert to black & white mode due to the missing color burst. This depends largely on the way the so-called “color killer” circuit (or algorithm) in the TV or monitor works. Some may act immediately when a scan line misses the color burst signal, others may take longer. This effect is largely unpredictable.

Yet another complication is that the video scanner of the Apple-1 motherboard works independently from the video scanner on the color graphics card – there is no “genlocking”. But the phase relationship never changes after power up.

Notes / hints to Arnaud:

I think it’s too soon to try to implement these complications in POM-1. The refresh cycles could be implemented easily by having a running counter which increments modulo 65 with every CPU cycle and when a refresh cycle strikes that cycle, then just increment <hpos> (with the rollover to 0, of course) used in the above algorithm before that cycle would do a memory read.

Depends on the 6502 engine you use in POM-1. Some 6502 emulators are cycle accurate but do not work on a cycle-by-cycle basis (they execute the instruction as a whole, and then advance a cycle counter by the number of cycles the instruction would need). This would NOT work with emulation of any “racing the beam” game systems like the Atari VCS (and the Atari 400+800 family) I am familiar with, so there are certainly 6502 emulators out there which work on a cycle-by-cycle basis.

Just as a side note, I have my own Apple-1 emulator, called “EXACTA-1” which is super accurate as it uses a switch-level simulation of the 6502 SPICE netlist which came out of the “Visual 6502” project in which I had a minor role as a NMOS technology consultant, and I helped them with explaining some of the weird circuits which they found from reverse engineering the silicon. I got the netlist in return ;-)

I have implemented the refresh cycles in EXACTA-1 and used it to develop the RWTS of my Apple-1 floppy disk controller. I also used it to develop the super fast ACI based routines which can load the BASIC in 2 seconds. But other than for these flights of fancy, EXACTA-1 is useless as it runs 10 times slower than the real Apple-1 does (on a modern notebook, mind you !). This is why I did not put a graphics card emulation into it. No way to develop games with moving objects when the emulator is that slow. This is why I am so excited about your upgraded POM-1 !

You also might want to implement the GEN1/GEN2 improved ACI in your POM-1. It just involves adding the $C5XX PROM page. I can send you the binary, if you want.

Keep up the good work !

And feel free to ask any questions !

- Uncle Bernie

(you can contact me at appleonedoc@gmail.com)